

# CSim Design Notes

Architecture, decisions, and the render-token migration

Project author + collaborative notes (Grok Build)

Started 2026-07-28 (living document)

## Abstract

This document is the **durable design memory** for CSim (Cell Simulator): what the code is doing today, why packages and abstractions exist (to the best of our knowledge), and the planned finish of the migration from immediate-mode OpenGL calls toward a **render command / token** architecture that abstracts the graphics API behind a backend.

It is intentionally incomplete in places. Sections marked as *inferred* were reverse-engineered from the tree; the author should overwrite them with intent. Use the decision log to keep choices stable across chat sessions.

## Contents

|          |                                                      |          |
|----------|------------------------------------------------------|----------|
| <b>1</b> | <b>How to use this document</b>                      | <b>3</b> |
| 1.1      | Purpose . . . . .                                    | 3        |
| 1.2      | Conventions . . . . .                                | 3        |
| 1.3      | Workflow for future sessions . . . . .               | 3        |
| 1.4      | Building the PDF . . . . .                           | 3        |
| <b>2</b> | <b>Project overview</b>                              | <b>4</b> |
| 2.1      | What CSim is . . . . .                               | 4        |
| 2.2      | High-level system diagram . . . . .                  | 4        |
| 2.3      | Runtime one-liner . . . . .                          | 4        |
| 2.4      | Non-goals (current) . . . . .                        | 4        |
| <b>3</b> | <b>Source layout</b>                                 | <b>5</b> |
| 3.1      | Related trees (not packages) . . . . .               | 5        |
| 3.2      | Naming landmine (Windows) . . . . .                  | 5        |
| 3.3      | Include style (current practice) . . . . .           | 5        |
| <b>4</b> | <b>Runtime loop and modules</b>                      | <b>5</b> |
| 4.1      | CellMain . . . . .                                   | 5        |
| 4.2      | Illumo as composition root . . . . .                 | 6        |
| 4.3      | IModule contract . . . . .                           | 6        |
| 4.4      | Frame flow (conceptual) . . . . .                    | 7        |
| <b>5</b> | <b>Rendering — current state</b>                     | <b>7</b> |
| 5.1      | What actually draws frames today . . . . .           | 7        |
| 5.2      | Drawable design (observed) . . . . .                 | 7        |
| 5.3      | Parallel / unfinished stacks still in tree . . . . . | 8        |

|           |                                                    |           |
|-----------|----------------------------------------------------|-----------|
| 5.4       | Backend surface (already sketched) . . . . .       | 8         |
| 5.5       | Pain points blocking the migration . . . . .       | 8         |
| <b>6</b>  | <b>Rendering — target token architecture</b>       | <b>9</b>  |
| 6.1       | Goals . . . . .                                    | 9         |
| 6.2       | Vocabulary . . . . .                               | 9         |
| 6.3       | Target frame pipeline . . . . .                    | 9         |
| 6.4       | Recommended layering (target) . . . . .            | 9         |
| 6.5       | Token payload design (proposal) . . . . .          | 10        |
| 6.6       | What happens to Drawable? . . . . .                | 10        |
| 6.7       | Migration phases (strangler) . . . . .             | 10        |
| 6.8       | Acceptance criteria for “migration done” . . . . . | 11        |
| 6.9       | Risks . . . . .                                    | 11        |
| <b>7</b>  | <b>Game domain and rulesets</b>                    | <b>11</b> |
| 7.1       | CellGameModule . . . . .                           | 11        |
| 7.2       | CellContext . . . . .                              | 12        |
| 7.3       | Canvas . . . . .                                   | 12        |
| 7.4       | Rulesets . . . . .                                 | 12        |
| <b>8</b>  | <b>Services, foundation, platform</b>              | <b>12</b> |
| 8.1       | Services . . . . .                                 | 12        |
| 8.2       | Foundation . . . . .                               | 13        |
| 8.3       | Platform . . . . .                                 | 13        |
| 8.4       | Third-party . . . . .                              | 13        |
| <b>9</b>  | <b>Design decision log</b>                         | <b>13</b> |
| <b>10</b> | <b>Open questions</b>                              | <b>17</b> |
| <b>11</b> | <b>Glossary</b>                                    | <b>18</b> |
| <b>A</b>  | <b>Build notes</b>                                 | <b>19</b> |
| A.1       | Configure & build (Windows) . . . . .              | 19        |
| A.2       | Known setup issues . . . . .                       | 19        |
| A.3       | House style (CONTRIBUTING) . . . . .               | 19        |
| <b>B</b>  | <b>Key file map</b>                                | <b>19</b> |
| B.1       | Archive . . . . .                                  | 19        |

# 1 How to use this document

## 1.1 Purpose

- Survive context resets (new chats, compacted histories).
- Record **why**, not only **what**.
- Host the render-token migration plan so implementation chats start from a shared baseline.
- Separate *observed reality* from *intended design*.

## 1.2 Conventions

### Status boxes

Snapshot of maturity (prototype, half-migrated, production-ish).

### Decision boxes

Closed choices. Prefer adding a new decision over rewriting history silently; note supersession dates.

### Open boxes

Questions only the author can settle.

### Inference boxes

Reverse-engineered claims. Treat as provisional.

### `\todoauthor{...}`

Explicit author TODO inline in the prose.

## 1.3 Workflow for future sessions

1. Skim section 2, section 5, section 6, and the decision log (section 9).
2. Implement or discuss against those sections, not against chat memory alone.
3. When a decision is made, append a dated entry to the decision log.
4. When code lands, update the “current state” sections and the file map.

## 1.4 Building the PDF

From the repo root (requires a TeX distribution with `pdflatex` or `latexmk`):

```
cd docs
pdflatex csim-design-notes.tex
pdflatex csim-design-notes.tex # second pass for refs/TOC
# or: latexmk -pdf csim-design-notes.tex
```

Output: `docs/csim-design-notes.pdf`.

### Status / provenance

As of 2026-07-28: source tree reorganized into packages (**App**, **Engine**, **Game**, ...); Release and Debug builds succeed on Windows (MSVC). Render path is still dual-architecture (immediate GL + unfinished command backend).

## 2 Project overview

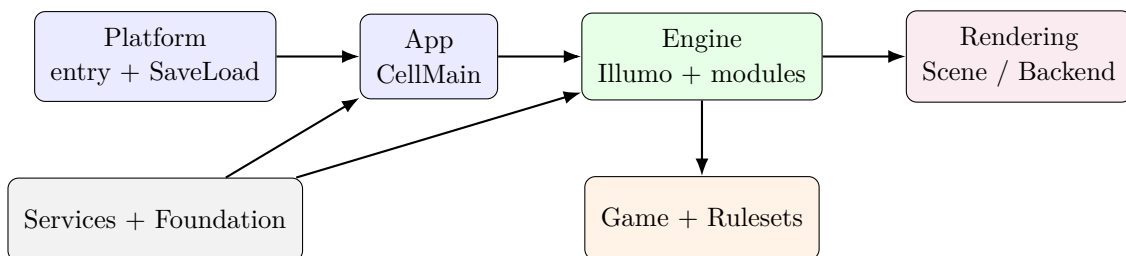
### 2.1 What CSim is

CSim is an interactive **cellular automata** (“cell”) simulator with a 2D canvas, multiple rule sets (Game of Life, Brian’s Brain, Wireworld, ...), camera navigation, edit/run modes, save/load hooks, and a small in-app command / debug surface. Rendering is currently OpenGL via GLFW/GLEW (with platform entry stubs for Linux/macOS).

### Inferred from code (correct if wrong)

Historically this grew from a simpler GoL program into a mini engine shell (“Illumo”): modules, env vars, input contexts, backend interfaces, Tracy hooks. The product goal remains a cell simulator; the engine shape is partly for skill practice and multi-API future-proofing.

### 2.2 High-level system diagram



### 2.3 Runtime one-liner

Platform `main` → CellMain → construct Illumo → Init (services + modules) → loop { Update, Render } until window close.

### 2.4 Non-goals (current)

#### Open / needs author input

Confirm or rewrite. Draft non-goals:

- Not a general-purpose AAA renderer.
- Not multiplayer / networking.
- Not a full ECS game engine (entity scaffolding exists but is thin).
- Perfect Linux/macOS parity is secondary to Windows correctness for now.

## 3 Source layout

Source lives under `CSim/Source/`. Package names describe **responsibility**, not prestige.

| Package     | Role                                                                                              |
|-------------|---------------------------------------------------------------------------------------------------|
| App/        | Process-level app loop after OS bootstrap ( <code>CellMain</code> ).                              |
| Engine/     | Host: <code>Illumo</code> , <code>IModule</code> , context bag, debug module, entity scaffolding. |
| Game/       | Domain: canvas, cell game module, editor tools (brush/cursor).                                    |
| Rulesets/   | Polymorphic CA rules; <code>AllSets.h</code> umbrella.                                            |
| Rendering/  | Window, camera, drawables, scene, RHI-ish interfaces, OpenGL backend.                             |
| Services/   | Log, env vars, input, CLI, allocators, <code>SaveLoad</code> API.                                 |
| Foundation/ | Macros, build/sys info, math type aliases, small containers.                                      |
| Assets/     | Asset loaders (e.g. TTF).                                                                         |
| Platform/   | OS entry points + native dialogs / save-load.                                                     |
| Tests/      | Early unit-test scratch (not fully wired into CMake).                                             |

### 3.1 Related trees (not packages)

- `CSim/Shader/` — GLSL assets copied to the build tree.
- `CSim/Assets/` — runtime files (fonts).
- `CSim/thirdparty/` — vendored deps (GLFW, GLEW, GLM, FreeType, Tracy, ...).
- `archive/` — historical code (old State pattern, analyzer dumps). Not built.

### 3.2 Naming landmine (Windows)

#### Decision: `MathTypes.h` (2026-07-28)

Do **not** put a header named `Math.h` on a case-insensitive include path. MSVC resolves `#include <math.h>` to it and breaks the CRT/GLM. Project math aliases live in `Foundation/MathTypes.h`.

### 3.3 Include style (current practice)

#### Status / provenance

CMake adds each package directory as an include root, so both `"Logger.h"` and `"Services/Logger.h"` can work. Prefer package-prefixed includes for new code so dependencies stay visible.

## 4 Runtime loop and modules

### 4.1 CellMain

`App/CellMain.cpp` owns logger init/shutdown and the frame loop:

```
// conceptual
Logger::initLogger();
Illumo* illumo = new Illumo(argc, argv);
```

```

illumo->Init();
while (!illumo->ShouldClose()) {
    double dt = /* glfw time delta */;
    illumo->Update(dt);
    illumo->Render();
}
illumo->Shutdown();

```

## 4.2 Illumo as composition root

Illumo owns the long-lived services (EnvVars, RenderWindow, Camera, Renderer, AssetManager, CommandRegistry, CommandLine, InputManager, Scene) and a vector of IModule instances.

IllumoContext is a **raw-pointer bag** into those services, passed into modules at **Start**. Ownership stays on Illumo.

### Inferred from code (correct if wrong)

Pattern: composition root + service locator-ish context struct (not a full service locator library). Modules are closer to subsystems than ECS systems.

## 4.3 IModule contract

```

class IModule {
public:
    virtual void Start(IllumoContext* context) = 0;
    virtual void Update(double dt) = 0;
    virtual void DispatchDrawables(Scene* scene) = 0;
    virtual void Exit() = 0;
};

```

- **Start** — bind inputs, allocate game state.
- **Update** — simulation / input.
- **DispatchDrawables** — contribute what should draw this frame.
- **Exit** — teardown module-owned state.

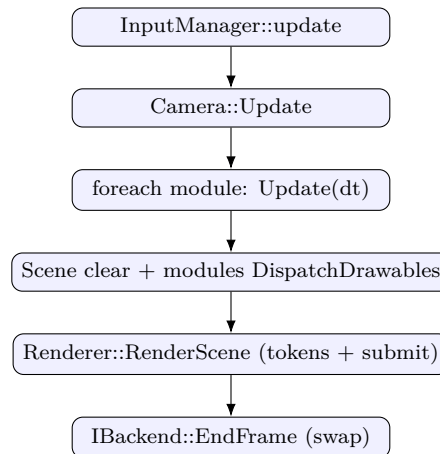
Registered modules today (from `Illumo::Init`):

- `CellGameModule` (always)
- `DebugModule` (non-NDEBUG)

### Open / needs author input

Engine header `Engine/Illumo.h` currently includes concrete `Game/CellGameModule.h`. Intended long-term: register modules from App or a bootstrap file so Engine does not depend on Game.

## 4.4 Frame flow (conceptual)



## 5 Rendering — current state

### Status / provenance

**Phases 1–6 complete for planned scope.** Live frame path is enroll + tokens + submit via `Renderer/IBackend`. Live drawables emit tokens; GL draw submission is confined to `Rendering/OpenGL/`. **Phase 6:** headless `Rendering/Mock/MockBackend.h`; `Renderer` accepts injected `IBackend*` (D-R8). `CTest MockBackend` covers unit + E2E (`RenderScene`, `Canvas` tokens, hybrid immediate, proof quad). No second real GPU API yet. Env `UseTokenProof=1` for visual debug.

### 5.1 What actually draws frames today

1. Modules call `DispatchDrawables` and add `DrawableBase*` pointers to `Scene`.
2. `Scene::Update` clears color/depth with raw `glClear*`, then calls `DrawableBase::Draw` on each entry.
3. Concrete drawables (e.g. `Canvas` via CRTP `Drawable<Canvas>`) run `DrawImpl`, which issues GL calls and owns GL object handles (VAO/VBO/textures/programs) on the drawable base.

### 5.2 Drawable design (observed)

- `DrawableBase` — type-erased base for storage; virtual `Draw`; destructor deletes GL objects.
- `Drawable<Derived>` — CRTP: virtual `Draw` dispatches to `Derived::DrawImpl` for static polymorphism after the first virtual hop.

### Inferred from code (correct if wrong)

CRTP here is a micro-optimization / style choice: keep a homogeneous list of bases while avoiding a second virtual call into each concrete drawer. Resource ownership on the base tightly couples “something that can be listed in a scene” to “OpenGL object lifetime,”

which fights multi-API abstraction.

### 5.3 Parallel / unfinished stacks still in tree

| Stack                      | Notes                                                                                     |
|----------------------------|-------------------------------------------------------------------------------------------|
| RenderQueue on window      | Older “queue of drawables” idea; add paths often commented out.                           |
| IBackend + CommandQueue    | Target direction; <code>Renderer::RenderScene</code> mostly clears the queue and returns. |
| EntityTable / ModuleObject | Handle-ish objects with mesh/texture IDs; not the primary cell path.                      |
| SceneObject hierarchy      | Parent/child transforms; lightly used relative to drawable list.                          |

### 5.4 Backend surface (already sketched)

IBackend exposes:

- lifecycle: `Initialize`, `Shutdown`, `BeginFrame`, `EndFrame`
- queue: `PushToCommandQueue`, `SubmitCommandQueue`, `ClearCommandQueue`
- resource creation returning `unsigned long` handles: mesh / shader / texture / descriptor set

`CommandType` already enumerates a useful token vocabulary: pipeline state, bindings, uniforms, render-pass ops, draw ops (`Rendering/RenderCommand.h`).

`Renderer` separates:

- **Enrollment** — create GPU resources up front (not mixed into per-frame command spam).
- **RenderScene** — intended to translate a scene into tokens and submit (stub today).

#### Intent already visible in code

Game/UI code should not call `gl*` directly long-term. It should either enroll resources or emit `RenderCommand` tokens; only the backend implementation speaks the API.

### 5.5 Pain points blocking the migration

1. `Canvas` is both domain grid and GL texture/drawable.
2. `DrawableBase` owns GL handles.
3. `Scene::Update` is a mini immediate renderer.
4. `RenderCommand` payload is a flat “kitchen sink” struct (not a tagged union / `std::variant`); many fields unused per command type.
5. Two drawable submission ideas (`Scene` list vs `RenderQueue`) without a single winner.



## 6 Rendering — target token architecture

This section is the **plan foundation** for finishing the migration. Treat it as the working design; amend via the decision log.

### 6.1 Goals

1. **API isolation:** only `Rendering/OpenGL/*` (and future backends) include `GL/Vulkan/Metal` headers for draw submission.
2. **Stable front-end:** game modules describe *what* to draw (meshes, textures, transforms, passes), not *how* in GL terms.
3. **Record then execute:** each frame builds a command stream (“tokens”), then the back-end walks it once.
4. **Resource handles:** CPU code holds opaque IDs; backend maps them to API objects.
5. **Incremental:** keep the simulator playable after each step; prefer strangler-fig migration over a big bang rewrite.

### 6.2 Vocabulary

#### Token / RenderCommand

A discrete instruction: bind X, set uniform Y, draw Z, begin pass, clear, ...

#### Command queue

Ordered buffer of tokens for the frame (or for a pass).

#### Backend

API-specific interpreter of tokens + owner of true GPU objects.

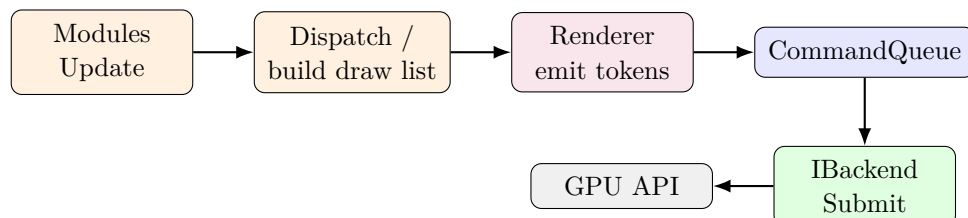
#### Enrollment

One-time (or rare) resource creation; returns a handle.

#### Front-end renderer

High-level helper that turns scene/draw requests into tokens (`Renderer` today).

### 6.3 Target frame pipeline



### 6.4 Recommended layering (target)

|                                       |
|---------------------------------------|
| Game / Rulesets / UI                  |
| (no gl* includes)                     |
| v                                     |
| High-level draw requests / scene data |

```

    | (handles, CPU buffers, transforms)
    v
Renderer -----enroll-----> IBackend::Create*
    | (push tokens)
    v
CommandQueue<RenderCommand>
    |
    v
IBackend::SubmitCommandQueue
    |
    +-- GLBackend
    +-- (future) other backends

```

## 6.5 Token payload design (proposal)

### Open / needs author input

Current `RenderCommand` is a single struct with many fields. Options:

1. **Keep flat struct** for simplicity and POD friendliness (array queue, Tracy, no heap). Accept waste.
2. **Tagged union** / `std::variant` for type safety and smaller conceptual surface.
3. **Byte stream + opcodes** (more engine-y; harder to debug).

Default recommendation for CSim scale: **(1) or a simple tagged union**, not a full GPU-driven byte compiler.

## 6.6 What happens to Drawable?

### Inferred from code (correct if wrong)

Long-term, “drawable” should mean *something that can append tokens* (or register a renderable descriptor), not *something that calls GL*. CRTP can remain for CPU-side specialization, but GL handle fields should move into the backend’s handle tables.

Proposed end state for `Canvas`:

- Domain: `lifeCanvas`, dimensions, cell get/set (no GL).
- View/render: texture handle + mesh handle + material/pass ids; upload dirty regions via enrollment or an `UpdateTexture` token.

## 6.7 Migration phases (strangler)

### 1. Phase 0 — Document & freeze vocabulary

This document; agree command enum + handle meaning; stop adding new raw `gl*` call sites outside OpenGL backend / temporary drawables.

### 2. Phase 1 — Make the queue real for one pass

Implement `GLBackend::SubmitCommandQueue` for a minimal set: `ClearColorBuffer`, `SetViewport`, `SetShader`, `SetTexture`, `SetUniformMat4`, `DrawIndexed`. Unit-test or visually verify a fullscreen clear + one textured quad *only* through tokens.

### 3. Phase 2 — Route Scene through Renderer

`Scene::Update` stops calling `glClear / Draw`. Instead: build tokens (or ask each renderable to append tokens), then `Renderer::RenderScene` → submit. Drawables may still be immediate behind a feature flag temporarily.

### 4. Phase 3 — Migrate Canvas

Split domain vs GPU resources; canvas upload becomes handle-based; `DrawImpl` becomes token emission.

### 5. Phase 4 — Migrate remaining drawables

Splash text, `GLString`, command-line overlay, debug module, cursor, ...

### 6. Phase 5 — Delete dead paths

Remove unused `RenderQueue`-as-drawable-list if tokens won; collapse duplicate transform types; ensure Engine does not include GL.

### 7. Phase 6 — Optional second backend

Only after OpenGL path is token-clean; e.g. mock backend for tests or a second real API.

#### Working order (proposed 2026-07-28)

Prefer **Phase 1 before large refactors of Game**. Prove the token interpreter with a tiny known scene, then move Canvas. Do not invent Vulkan until OpenGL is fully behind `IBackend`.

## 6.8 Acceptance criteria for “migration done”

- No `gl*` / GLEW includes outside `Rendering/OpenGL/` (and maybe temporary test helpers).
- Frame submission path is enroll + token queue + submit.
- Cell sim still runs: edit, step/rules, camera, save/load hooks.
- `Renderer::RenderScene` is non-empty and used.
- Decision log updated; dead stacks removed or quarantined under `archive/`.

## 6.9 Risks

- Scope creep into full RHI (descriptor sets, multi-pass graph) before the sim needs them.
- Handle lifetime / deletion protocol undefined today.
- Performance paranoia too early (2048 command cap may be fine).
- Platform divergence (Linux/macOS files still older in places).

# 7 Game domain and rulesets

## 7.1 CellGameModule

Primary gameplay module. Owns a `CellContext` and an enum `CellState`:

**Status / provenance**

Mode handling is an **enum** + **switch** inside the module, not the older class-per-state hierarchy (archived under [archive/old-states/](#)). SAVE/LOAD cases may still be incomplete relative to method helpers.

**Inferred from code (correct if wrong)**

Earlier design used State classes so each mode had its own “main loop” body. The enum approach is simpler for a small mode set; reintroduce State objects only if modes grow heavy (UI stacks, async load, etc.).

## 7.2 CellContext

Constructs `Canvas` from env vars (`CanvasX`, `CanvasY`) and selects a `RuleSet` from a mode string (`GAME_OF_LIFE`, `BRIANS_BRAIN`, ...).

## 7.3 Canvas

- CPU grid: `lifeCanvas` (and RGB staging `texCanvasBuffer`).
- Currently also a `Drawable` with GL texture id — migration target is to split this (section 6).

## 7.4 Rulesets

Classic polymorphic hierarchy:

```
class RuleSet {
    virtual void evalCell(...) const;
    virtual void evaluateNeighbors(...) const;
    void calcGeneration(...); // shared sweeping
};
```

Concrete rules live in `Rulesets/`. This package is relatively clean: depends on `canvas/domain`, not on windowing.

**Keep rules pure**

Rulesets should stay free of rendering and input. They read/write cell buffers only. That keeps them testable and migration-safe.

# 8 Services, foundation, platform

## 8.1 Services

Cross-cutting facilities used by Engine/Game/Rendering:

- **Logging** — `Logger` (file + terminal; level from env).

- **Config** — `EnvVars` / `IEnvVars` (defaults filled in `Illumo::Init`: canvas size, fps/tps, mode string, ...).
- **Input** — `InputManager`, `InputContext`, `KeyCode`; action binding used by the game module.
- **CLI** — `CommandLine`, `CommandRegistry`, `SysCmdLine`.
- **Allocators** — pool/arena/debug allocators (experimental / learning surface; not all wired into the hot path).
- **SaveLoad API** — pure declarations; OS dialogs implemented per platform.

## 8.2 Foundation

Headers that should stay cheap to include: macros (`MacroDefs.h`), build info, `sysinfo`, `MathTypes.h`, `ArrayQueue.h`.

### Open / needs author input

`MacroDefs.h` pulls Windows headers on `_WIN32`. That is convenient for `Sleep` etc., but increases include toxicity. Consider splitting platform sleep helpers out of the ubiquitous macro header.

## 8.3 Platform

Thin ports:

- Windows: `WinMain.cpp`, `WinSaveLoad.cpp`
- Linux: `_main.cpp`, `LinuxSaveLoad.cpp` (GTK pieces may be legacy relative to GLFW-centered shared code)
- macOS: `Main.cpp`, `MacSaveLoad.mm`

Contract: bootstrap → call `CellMain` → implement `SaveLoad::GetLoadLocation` / `GetSaveLocation`.

## 8.4 Third-party

Vendored under `CSim/thirdparty/`. Policy from `CONTRIBUTING.md`: new third-party deps require review/PR. Existing: GLFW, GLEW, GLM, FreeType, JSON, STB, Tracy, image libs, ...

# 9 Design decision log

Append-only log. Newest entries at the bottom. If a decision is reversed, add a new entry that supersedes the old id; do not silently delete history.

## D-001 — Package rename (structure)

**D-001** 2026-07-28

**Decision:** Rename/split ambiguous folders to match responsibility: `Core`→`Game`, `System`→`{App,Engine,Services}`, `Init+Util`→`Foundation`, `AssetLoaders`→`Assets`.

**Why:** Names were lying; slowed reasoning and migrations.

**Consequences:** Include/CMake paths updated; archive for old States.

## D-002 — MathTypes header name

**D-002** 2026-07-28

**Decision:** Project math aliases live in `MathTypes.h`, not `Math.h`.

**Why:** Windows case-insensitive include collision with CRT `math.h`.

**Consequences:** All includes updated; Foundation README notes the trap.

## D-003 — Render abstraction direction

**D-003** 2026-07-28 (intent; implementation incomplete)

**Decision:** Move toward command/token submission via `IBackend` + `RenderCommand` queue; game code must not permanently depend on raw GL.

**Why:** Multi-API optionality; clearer frame boundary; testability.

**Consequences:** Parallel stacks exist until migration phases complete (section 6.7).

## D-004 — Module frame hooks

**D-004** inferred from `IModule`

**Decision:** Modules expose `Start/Update/DispatchDrawables/Exit`.

**Why:** Separate simulation tick from draw contribution.

**Consequences:** Draw list is rebuilt or contributed each frame through dispatch (exact ownership policy still open).

## D-005 — Rules as polymorphic sets

**D-005** inferred

**Decision:** Each CA variant is a `RuleSet` subclass.

**Why:** Open for extension; shared generation sweep.

**Consequences:** Factory logic currently in `CellContext` string matching (could become a registry later).

## D-006 — Mode handling (enum vs State classes)

**D-006**    current code; author may supersede

**Decision (current):** `CellState` enum inside `CellGameModule`.

**Supersedes:** class hierarchy under `archive/old-states/`.

**Why (inferred):** fewer types for a small mode set.

**Open:** SAVE/LOAD completeness; whether splash/global state is acceptable.

## D-007 — Enrollment vs per-frame commands

**D-007**    visible in `Renderer` comments

**Decision:** Resource creation goes through `enroll/Create*` APIs; do not stuff mesh/shader creation into the per-frame token stream.

**Why:** Keep the frame queue about *binding and drawing*.

**Consequences:** Need an explicit destroy/reload story for hot-reload later.

## D-008 — CONTRIBUTING constraints

**D-008**    from `CONTRIBUTING.md`

**Decision:** Avoid `auto`; avoid namespaces (prefer static classes/structs); no recursion; third-party via PR.

**Why:** Author house style / learning constraints.

**Consequences:** Documentation and new code should respect these unless explicitly waived in a later decision.

## D-R1 — `RenderCommand` payload shape

**D-R1**    2026-07-28

**Decision:** `RenderCommand` uses a **simple tagged union** (`CommandType` + POD payload union), not a kitchen-sink flat struct and not a byte-stream compiler. Prefer C-style union over `std::variant`.

**Why:** Type-safe uniforms/clear/update data while remaining copyable for `ArrayQueue`; matches house style.

**Consequences:** `Rendering/RenderCommand.h` redesigned; execute path switches on type and reads the matching payload.

## D-R2 — Who emits tokens

**D-R2**    2026-07-28

**Decision:** Each drawable emits its own tokens via `AppendCommands(Renderer*)` (or equivalent). `Renderer` owns frame setup (clear/viewport) and submit.

**Why:** Smallest strangler step from today's `DispatchDrawables` list.

**Consequences:** Hybrid phase may still call immediate `Draw` for unmigrated drawables after token submit.

## D-R3 — Handle typing for v1

**D-R3** 2026-07-28

**Decision:** Keep unsigned long opaque handles with backend registries (`tableID` → GL object). Strong typedefs deferred.

**Why:** Minimal churn; critical fix is resolve-on-submit, not type names.

**Consequences:** Execute must never treat handles as raw GLuints without registry lookup.

## D-R4 — Migration scope

**D-R4** 2026-07-28

**Decision:** Full migration Phases 1–5 (token proof → Scene routing → Canvas → UI drawables → cleanup). Phase 6 (mock/second API) optional later.

**Why:** Playable strangler path to API isolation without big-bang rewrite.

**Also freeze for v1:** destroy resources at shutdown only; full Canvas texture rewrite each frame; Windows GL is migration target.

## D-R5 — Blend state on first enable

**D-R5** 2026-07-28

**Decision:** When enabling blending in `GLDevice::ApplyPipelineState`, always call `glBlendFunc` (do not rely on CPU-side factor cache matching defaults).

**Why:** OpenGL default is `ONE/ZERO`; cache defaults were `SrcAlpha/OneMinusSrcAlpha` so factors looked “unchanged” and console panel lost transparency after the token migration.

**Consequences:** Semi-transparent UI (console panel alpha) works again.

## D-R6 — Dead stack cleanup (Phase 5)

**D-R6** 2026-07-28

**Decision:** Archive window `RenderQueue`, legacy `Drawable.cpp` GL helpers, empty `Transform/RenderContext`, and unused `CellCommandLine` under `archive/dead-render/`. `DrawableBase` is a pure list/token interface (no GL handles).

**Why:** Token path won; dual submission models confuse maintenance.

**Consequences:** `IRenderWindow::getRenderQueue` removed; frame draw list is only `Scene + Renderer`.

## D-R7 — Mock backend for tests (Phase 6)

**D-R7** 2026-07-28

**Decision:** Add headless `MockBackend` implementing `IBackend`: record `Create*` and snapshot the command queue on `SubmitCommandQueue`. Ship as CMake target `CSimRenderTests / CTest MockBackend`. Do **not** add Vulkan/Metal yet.

**Why:** First meaningful automated render test without a GPU context; proves token



vocabulary and handle plumbing independently of GL.

**Consequences:** CI can run `ctest -R MockBackend`; second real API remains future work after more production needs appear.

## D-R8 — Inject IBackend into Renderer

**D-R8 2026-07-28**

**Decision:** Renderer accepts an injected `IBackend*` via constructor (`takeOwnership` flag). Production still owns a `GLBackend`. Tests inject `MockBackend` with `takeOwnership=false`.

**Why:** End-to-end drawable → enroll/token → submit without a GPU; keeps production construction unchanged.

**Consequences:** `TestRendererE2E` covers `RenderScene`, Canvas tokens, hybrid immediate drawables, and `RenderProofQuad`.

### Template for new entries

```
\subsection*{D-0XX --- short title}
\begin{decisionbox}[title={D-0XX \quad YYYY-MM-DD}]
\textbf{Decision:} ...
\textbf{Why:} ...
\textbf{Consequences:} ...
\textbf{Supersedes:} D-0YY (optional)
\end{decisionbox}
```

## 10 Open questions

Use this as a punch list when bouncing ideas. Strike through or move to the decision log when resolved.

### Open / needs author input

1. ~~**Token payload:** flat struct vs variant vs byte stream?~~ **Resolved D-R1:** simple tagged union (C-style).
2. ~~**Handle type:** keep unsigned long ...?~~ **Resolved D-R3:** unsigned long + registries for v1.
3. ~~**Who emits tokens?** each drawable vs scene compiler?~~ **Resolved D-R2:** each drawable emits via `AppendCommands`.
4. **Resource ownership:** refcounts, generational handles, or “destroy only at shutdown” for v1? (*Working answer D-R4: shutdown-only for v1.*)
5. **Canvas upload:** full texture rewrite each frame vs dirty rects? (*Working answer D-R4: full rewrite for v1.*)
6. **Illumo ↔ Game dependency:** how should modules be registered without Engine including Game headers?
7. **InputManager → CellContext include:** should Services depend on Game at all?

8. **RenderQueue class:** delete, rename, or repurpose as the token queue UI name?
9. **EntityTable:** first-class for cells, only for 3D-ish objects, or experimental dead code?
10. **Linux/macOS:** invest now or freeze until token GL path is solid on Windows?  
(*Working answer D-R4: freeze until Windows token path solid.*)
11. **Tracy:** always-on in Debug only — any CI policy?
12. ~~Tests: first meaningful automated test?~~ **Resolved D-R7:** mock backend submit sequence (`CsimRenderTests` / `CTest MockBackend`).
13. **Author intent name “Illumo”:** engine codename meaning / permanence?

## 11 Glossary

### Backend

Implementation of `IBackend` for a graphics API.

### Canvas

Cell grid (+ currently also its GPU presentation).

### Command / token

One entry in the render command queue describing a GPU operation.

### CRTP

Curiously Recurring Template Pattern; used by `Drawable`.

### Drawable

Object that can be presented; today immediate GL, tomorrow token emitter.

### Enrollment

Creating GPU resources and receiving handles outside the tight frame loop.

### Illumo

Engine/host object: owns services, modules, frame pumping.

### IllumoContext

Non-owning pointers to services for modules.

### Module

`IModule` subsystem (game, debug, ...).

### RuleSet

Cellular automaton transition rules for the canvas.

### Scene

Currently a list of drawables + optional node graph; should become data for the renderer, not an immediate GL executor.

## A Build notes

### A.1 Configure & build (Windows)

```
cmake -S CSim -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Release --target CSim
cmake --build build --config Debug --target CSim
```

Artifacts:

- build/Release/CSim.exe
- build/Debug/CSim.exe

### A.2 Known setup issues

- GLEW is pulled via `thirdparty/glew-2.1.0/build/cmake`; that subtree must exist (do not delete vendored CMake helpers).
- Debug enables `/fsanitize=address`; requires a toolchain that supports MSVC ASan.
- FreeType may warn about missing optional system deps (HarfBuzz, zlib, PNG); build can still succeed with bundled defaults.

### A.3 House style (CONTRIBUTING)

See `CSim/CONTRIBUTING.md`: limited `auto`, avoid namespaces, no recursion, third-party via PR.

## B Key file map

Quick index for chats that need to open the right files. Not exhaustive.

### B.1 Archive

- `archive/old-states/` — previous State pattern experiments.
- Analyzer/build junk may also live under `archive/`; ignore for design.

| Path                         | Why it matters                        |
|------------------------------|---------------------------------------|
| App/CellMain.*               | App loop                              |
| Engine/Illumo.*              | Composition root                      |
| Engine/IllumoContext.h       | Service pointer bag                   |
| Engine/IModule.h             | Module API                            |
| Engine/DebugModule.*         | Debug overlay module                  |
| Game/CellGameModule.*        | Modes, input, sim tick                |
| Game/CellContext.h           | Canvas + ruleset factory              |
| Game/Canvas.*                | Grid + (today) GL drawable            |
| Rulesets/RuleSet.*           | Rule base + generation                |
| Rendering/Scene.h            | Drawable list + immediate clear/draw  |
| Rendering/Drawable.*         | CRTP drawable / GL resources          |
| Rendering/RenderCommand.h    | Token enum + struct                   |
| Rendering/CommandQueue.*     | Token buffer                          |
| Rendering/IBackend.h         | Backend interface                     |
| Rendering/Renderer.h         | Enroll + (stub) RenderScene           |
| Rendering/OpenGL/GLBackend.* | OpenGL backend                        |
| Rendering/IRenderWindow.h    | Window abstraction                    |
| Services/InputManager.*      | Input contexts/actions                |
| Services/EnvVars.*           | Runtime config                        |
| Services/Logger.*            | Logging                               |
| Services/SaveLoad.h          | Ported dialogs API                    |
| Platform/Windows/*           | Win main + file dialogs               |
| Foundation/MathTypes.h       | GLM aliases (not Math.h!)             |
| Foundation/MacroDefs.h       | Force-inline, sleep, Windows includes |
| CSim/CMakeLists.txt          | Sources + include roots               |